

Development Guide — Mobile Application (Flutter)

Cross-platform mobile app (Android/iOS/Web/Windows) built with Flutter. Backend is developed and deployed separately; this app is a pure API consumer.

This guide documents the **actual current state** of `app/lib` step by step, in the order you'd rebuild it from scratch. Every file path below exists in the repo — open it alongside this guide rather than copy-pasting blindly.

1. Architecture

Pattern: Feature-first + Clean Architecture (`data` / `domain` / `presentation` per feature). Backend API changes stay isolated to each feature's `data` layer.

Concern	Choice	Why
Structure	Feature-first + Clean Architecture	Scales cleanly as features grow
State management	Riverpod (<code>StateNotifierProvider</code>)	Compile-safe DI, testable, no <code>BuildContext</code> ceremony
Networking	Dio + interceptors	Auth token injection, error mapping, logging in one place
Routing	go_router with <code>redirect</code> + <code>refreshListenable</code>	Declarative auth-guarded navigation
Models	freezed + json_serializable	Immutable models, generated <code>toJson</code> / <code>fromJson</code> / <code>copyWith</code>
Local storage	flutter_secure_storage (tokens, remember-me flag)	Tokens never sit in plain prefs
Env config	<code>--dart-define</code>	One codebase, dev/staging/prod backend URLs
Typography	google_fonts (Poppins)	Premium type scale without bundling font files
Micro-animations	flutter_animate + hand-rolled <code>AnimationController</code> s	Chained fade/slide/scale without Lottie assets
Biometrics	local_auth	Face ID / fingerprint unlock, mobile-only

2. Prerequisites

```
flutter --version  
flutter doctor
```

Resolve anything `flutter doctor` flags (Android toolchain, iOS toolchain, editor plugin) before continuing. On Windows, enable **Developer Mode** (`start ms-settings:developers`) — Flutter needs symlink support to build with plugins.

3. Project location

The Flutter project lives in `app/`, not the repo root:

```
cd D:\Testing\projects\MobileApplication\app  
flutter pub get
```

4. Folder structure

```
lib/
├── main.dart                                # runApp(ProviderScope(child: MyApp()))
├── app.dart                                # MaterialApp.router + theme wiring
├── core/
│   ├── config/
│   │   └── env.dart                        # API base URL via --dart-define
│   ├── network/
│   │   ├── dio_client.dart                # Dio + auth-token interceptor
│   │   └── api_exception.dart
│   ├── router/
│   │   └── app_router.dart                # go_router, auth redirect guard
│   ├── storage/
│   │   └── secure_storage.dart            # token + remember-me flag
│   ├── services/
│   │   └── biometric_service.dart         # local_auth wrapper
│   ├── theme/
│   │   ├── app_colors.dart                # palette + glass fill/border helpers
│   │   ├── app_text_styles.dart           # Poppins TextTheme
│   │   └── app_theme.dart                 # Material 3 light/dark ThemeData
│   └── widgets/
│       ├── animated_gradient_background.dart # drifting blurred gradient orbs
│       ├── glass_card.dart                 # BackdropFilter glassmorphism card
│       ├── gradient_button.dart            # primary CTA, press/loading animation
│       ├── premium_text_field.dart         # floating-label field w/ validation
│       ├── social_login_button.dart        # shared glass social button shell
│       └── brand_glyphs.dart               # Google/Apple/Microsoft placeholder marks
├── features/
│   ├── auth/
│   │   ├── data/
│   │   │   ├── models/
│   │   │   │   └── login_response_model.dart # @freezed, maps the flat /Auth/login response
│   │   │   └── repositories/
│   │   │       └── auth_repository.dart      # login/logout + social-login stub
│   │   ├── domain/
│   │   │   ├── entities/
│   │   │   │   └── user.dart                # plain business object
│   │   └── presentation/
│   │       ├── providers/
│   │       │   ├── auth_provider.dart        # AuthState/AuthNotifier
│   │       │   └── login_form_provider.dart  # form field state + validation
│   │       └── screens/
│   │           └── login_screen.dart          # the premium login UI
│   ├── home/
│   │   └── presentation/
│   │       └── screens/
│   │           └── home_screen.dart          # post-login KPI dashboard
│   └── shell/
│       ├── presentation/
│       │   ├── nav_items.dart               # NavItem list - single source of truth for
│       │   ├── screens/app_shell.dart       # responsive frame: side rail / bottom nav
│       │   └── widgets/
│       │       └── premium_side_menu.dart    # wide-layout side rail
```

```

└── bottom_nav_bar.dart          # narrow-layout bottom bar + scan FAB
└── shipments/
    ├── domain/
    │   └── shipment.dart        # ShipmentStatus enum + Shipment/JourneyStep
    └── presentation/
        ├── screens/
        │   ├── shipment_list_screen.dart
        │   ├── shipment_detail_screen.dart
        │   └── scan_screen.dart
        └── widgets/shipment_widgets.dart  # StatusChip + RouteProgress
└── notifications/
    └── presentation/
        └── screens/alerts_screen.dart    # AppNotification model + list UI
└── profile/
    └── presentation/
        └── screens/profile_screen.dart

```

Every new feature (e.g. a future `settings`) follows the same `data` / `domain` / `presentation` split as `auth` , even if `data` / `domain` start out empty — `home` , `shell` , `shipments` , `notifications` , and `profile` above are all at some earlier stage of that same split, since none of them call a real backend yet.

5. Dependencies

`pubspec.yaml` (dependencies added beyond `flutter create` defaults):

```

dependencies:
  flutter_riverpod: ^2.5.1
  go_router: ^14.2.7
  dio: ^5.6.0
  freezed_annotation: ^2.4.4
  json_annotation: ^4.9.0
  flutter_secure_storage: ^9.2.2
  shared_preferences: ^2.3.2
  google_fonts: ^6.2.1
  flutter_animate: ^4.5.0
  local_auth: ^2.3.0

dev_dependencies:
  build_runner: ^2.4.13
  freezed: ^2.5.7
  json_serializable: ^6.8.0
  flutter_lints: ^6.0.0

```

```
flutter pub get
```

6. Core layer

Build these first — every feature depends on them.

6.1 `lib/core/config/env.dart`

Backend base URL, injected at build/run time — never hardcoded.

```
class Env {
  static const String apiUrl = String.fromEnvironment(
    'API_BASE_URL',
    defaultValue: 'https://localhost:7161/api', // local backend; Swagger UI at /swagger/index.html
  );
}
```

Default points at a local backend reachable directly from Chrome/Windows desktop. Running on an Android **emulator** instead needs the host-loopback address:

```
flutter run --dart-define=API_BASE_URL=http://10.0.2.2:7161/api
```

If the backend uses the ASP.NET Core dev HTTPS certificate, trust it once with `dotnet dev-certs https --trust` on the host machine, or requests will fail with a certificate error.

6.2 `lib/core/storage/secure_storage.dart`

Stores the auth token **and** the remember-me flag, both outside plain `shared_preferences`.

6.3 `lib/core/network/api_exception.dart` + `dio_client.dart`

`DioClient` auto-attaches `Authorization: Bearer <token>` to every request via an interceptor, and maps Dio errors to `ApiException`.

6.4 `lib/core/theme/`

Three files, built in this order:

1. `app_colors.dart` — brand palette (`primary #2563EB`, `secondary #4F46E5`, `accent #06B6D4`, semantic success/warning/error) plus `glassFill()` / `glassBorder()` helpers that adapt to light/dark brightness.
2. `app_text_styles.dart` — wraps `GoogleFonts.poppinsTextTheme()` and pins the sizes from the spec (32/24/16/14, weights 300–700).
3. `app_theme.dart` — builds `ThemeData` for both brightnesses: Material 3, rounded (`radius 20`) input/button shapes, `FadeForwardsPageTransitionsBuilder` on Android and `CupertinoPageTransitionsBuilder` on iOS.

`CupertinoPageTransitionsBuilder` lives in `package:flutter/cupertino.dart`, not `material.dart` — import it explicitly or `flutter analyze` fails.

6.5 `lib/core/widgets/`

Reusable, theme-aware building blocks used by every auth-style screen:

- `animated_gradient_background.dart` — a looping `AnimationController` drifts three `RadialGradient` orbs behind the content (cheap alternative to a real blurred bitmap — no `BackdropFilter` cost paid every frame for the backdrop itself).
- `glass_card.dart` — the actual glassmorphism: `BackdropFilter(ImageFilter.blur)` inside a `ClipRRect`, translucent fill + border from `AppColors.glassFill/Border`.
- `gradient_button.dart` — primary CTA: gradient fill, press-scale animation, `AnimatedSwitcher` between label and spinner for the loading state.
- `premium_text_field.dart` — floating label via `InputDecoration`, leading icon, optional suffix (used for the password visibility toggle), `errorText` wired to external validation state.
- `social_login_button.dart` — shared glass-pill shell for the OAuth row; takes any `icon` widget so it isn't tied to a specific provider.
- `brand_glyphs.dart` — **placeholder** Google/Apple/Microsoft marks (styled "G", `Icons.apple`, a 4-color grid) so the social row doesn't need bundled logo assets. Swap for official SVG/PNG brand marks before shipping.

6.6 `lib/core/services/biometric_service.dart`

Thin wrapper over `local_auth`. `isSupportedPlatform` is `false` on web/desktop so callers can hide the biometric button there without a runtime exception.

Biometrics requires platform wiring — already done in this repo:

- **Android:** `android/app/src/main/AndroidManifest.xml` has `<uses-permission android:name="android.permission.USE_BIOMETRIC" />`, and `MainActivity.kt` extends `FlutterFragmentActivity` (not `FlutterActivity` — `local_auth` requires the Fragment variant).
- **iOS:** `ios/Runner/Info.plist` has `NSFaceIDUsageDescription`.

6.7 `lib/core/router/app_router.dart`

`routerProvider` builds a `GoRouter` with a `redirect` callback gated on `authProvider.isAuthenticated`, driven by a `_AuthRefreshNotifier` (a `ChangeNotifier` that calls `notifyListeners()` whenever `ref.listen(authProvider, ...)` sees the auth state flip) passed as `refreshListenable`. Unauthenticated users get bounced to `/login`; authenticated users hitting `/login` get bounced to `/home`.

7. Auth feature

7.1 `lib/features/auth/domain/entities/user.dart`

Plain `User { id, name, groupId, empCode, description }` — no JSON, no Riverpod, no Dio. Keep it that way. `id` is **not** part of the backend response (see below) — it's the user ID the person typed into the login form, threaded through by the repository.

7.2 `lib/features/auth/data/models/login_response_model.dart`

`@freezed LoginResponseModel` mirrors the actual `/api/Auth/login` response, which is flat (no nested `user` object) and doesn't include an `id` or `email`:

```
{
  "success": true,
  "message": "Administrator",
  "token": null,
  "userName": "SHAHNAWAZ FARIDI",
  "userGroupId": "OPNUSER1",
  "userEmpCode": "IS4041",
  "userDesc": "Administrator",
  "status": "TRUE"
}
```

`token` is nullable and typically `null` today — this backend doesn't issue a bearer token on login yet. `toEntity(loginUserId)` takes the form's typed user ID as a parameter since the response has nothing to use as a stable identifier. Requires code generation — see §9.

7.3 `lib/features/auth/data/repositories/auth_repository.dart`

- `login({userId, password, rememberMe})` → `POST /Auth/login` (full path `/api/Auth/login` once joined with `Env.apiUrl`). Success/failure is signaled by the response's `success` field, **not** HTTP status — a `false` value throws `ApiException(message)`. Token is only persisted if the response actually includes one (see §7.2 — currently it doesn't).
- `loginWithProvider(SocialProvider provider)` → **throws** `UnimplementedError` with a message explaining why (no OAuth client IDs / backend token-exchange endpoint configured yet). This is intentional — the UI is real, the wiring isn't.
- `readStoredToken()` / `logout()`.

7.4 `lib/features/auth/presentation/providers/auth_provider.dart`

- Provider plumbing: `secureStorageProvider` → `dioClientProvider` → `authRepositoryProvider`, plus `biometricServiceProvider`.
- `AuthState { isLoading, user, error }` with `isAuthenticated` getter (drives the router redirect).

- `AuthNotifier` : `login()` , `loginWithProvider()` , `loginWithBiometrics()` (re-auths against an already-stored token — biometrics unlock a session, they don't replace the first password login), `logout()` .

7.5 `lib/features/auth/presentation/providers/login_form_provider.dart`

`LoginFormState` holds raw field values plus `userIdTouched` / `passwordTouched` flags so errors only appear after a field has been interacted with (not on first paint).

`userIdError` / `passwordError` / `isValid` are derived getters — plain user ID pattern (`3-20` chars: letters, numbers, `.` or `_` , **not** email format), password just required non-empty (no length rule — the backend doesn't enforce one). `loginFormProvider` is `.autoDispose` so form state resets if the user navigates away and back.

7.6 `lib/features/auth/presentation/screens/login_screen.dart`

Assembles everything above: `AnimatedGradientBackground` → centered `ConstrainedBox(maxWidth: 440)` (this is what makes it responsive on tablet/web without a separate layout) → logo + heading (animated in with `flutter_animate`) → `GlassCard` containing the form → remember-me/forgot-password row → `GradientButton` → conditional biometric icon button → register/legal/version footer.

The "OR CONTINUE WITH" divider and the `SocialLoginButton` row are commented out (not deleted) in this file until real OAuth is wired — see §12. Their imports (`brand_glyphs.dart` , `social_login_button.dart` , `auth_repository.dart` for `SocialProvider`) are commented out alongside them to keep `flutter analyze` clean.

`ref.listen(authProvider, ...)` drives snackbars for login errors and success — side effects live in `listen` , not in `build()` .

8. Home dashboard feature

`lib/features/home/presentation/screens/home_screen.dart` — `ConsumerWidget` that `ref.watch` es `authProvider` for the greeting name and renders a fixed `ListView` : header (greeting + gradient-avatar initials via `core/utils/initials.dart`), a 2×2 KPI `GridView` (active shipments / in transit / at customs / delivered today — still hardcoded `'0'` placeholders, not wired to a real summary endpoint), and up to two "active container" cards built from a `shipments` list passed in via constructor (empty by default — see §10, this screen has no data source of its own).

9. Shell / navigation feature

`lib/features/shell/` gives every authenticated screen a consistent header and nav, instead of each screen building its own `Scaffold` / `AppBar` /logout button. See `docs/plans/premium-navigation-side-menu.md` for the original design rationale.

- `nav_items.dart` — the single `List<NavItem>` (icon/selectedIcon/label/path) that both nav widgets below render, and that must stay in sync with the routes declared in `app_router.dart`.
- `app_shell.dart` — `AppShell(child, currentPath)` picks between `PremiumSideMenu` (width $\geq$ `_wideBreakpoint = 900.0`, via `MediaQuery.sizeOf(context).width`) and `AppBar` (narrower) at a single breakpoint, so resizing the window swaps chrome live. Wrapped around every authenticated route by a `ShellRoute` in `app_router.dart`, so it (and the side menu / bottom bar) stays mounted across in-app navigation instead of rebuilding on every screen change.
- `widgets/premium_side_menu.dart` — glassmorphic side rail: avatar header, nav list (staggered `flutter_animate` entrance), logout tile pinned below a divider.
- `widgets/bottom_nav_bar.dart` — glass bottom bar with a centered gradient scan FAB (opens `ScanScreen`) between the first two and last two nav items.

10. Shipments feature

`lib/features/shipments/` — container/shipment tracking UI.

- `domain/shipment.dart` — `ShipmentStatus` enum (`onVessel` / `atCustoms` / `inTransit` / `delayed` / `delivered`) with an extension (`ShipmentStatusX`) providing `.label` / `.color` / `.icon` getters, plus the `Shipment` and `JourneyStep` entities. No `data/` layer yet — nothing calls a real shipments API, so every screen below receives its data as a plain constructor parameter (empty lists today, supplied by `app_router.dart`). Add `data/repositories/shipment_repository.dart` + a provider here, mirroring `features/auth/`, once a backend endpoint exists.
- `presentation/screens/shipment_list_screen.dart` — searchable/filterable list (`StatefulWidget`, local `_filter` index into `['All', 'Ocean', 'Customs', 'Road']`), matching mockup 3b.
- `presentation/screens/shipment_detail_screen.dart` — vertical customs/journey timeline built from `Shipment.journey`, matching mockup 3c.
- `presentation/screens/scan_screen.dart` — barcode/Bayan-QR scan UI, matching mockup 3e. **The camera view (`_ScannerFrame`) is a stubbed placeholder** — an animated gradient box with a sweeping scan-line, not a real camera feed. Wire in a plugin such as `mobile_scanner` when integrating an actual scanner.
- `presentation/widgets/shipment_widgets.dart` — `StatusChip` and `RouteProgress`, shared by the home dashboard, list, and detail screens.

11. Notifications feature

`lib/features/notifications/presentation/screens/alerts_screen.dart` — alerts feed matching mockup 3f. `AppNotification` is defined inline in this file (no `domain/` folder yet, same reasoning as shipments). `AlertsScreen` takes a `notifications` list via constructor, empty by default; highlighted cards (`AppNotification.highlight`) get an accent-tinted border instead of the standard `SoftCard` look.

12. Profile feature

`lib/features/profile/presentation/screens/profile_screen.dart` — signed-in account screen (`ConsumerWidget` reading `authProvider` 's `user`): avatar/name/ description, employee code + user group info rows, and a log-out row. This is the **only place to log out on the narrow/mobile layout**, since `AppBar` has no logout button (wide layouts also have one in `PremiumSideMenu`).

13. App entry point

`lib/app.dart`

`MyApp` is a `ConsumerWidget` (not `StatelessWidget` , because it needs `ref.watch(routerProvider)`). Wires `AppTheme.light` / `.dark` with `themeMode: ThemeMode.system` .

`lib/main.dart`

```
void main() {  
  runApp(const ProviderScope(child: MyApp()));  
}
```

14. Code generation and run

`LoginResponseModel` needs generated `.freezed.dart` / `.g.dart` files before the app compiles:

```
dart run build_runner build --delete-conflicting-outputs  
flutter run -d chrome --web-port=5001
```

Re-run `build_runner build --delete-conflicting-outputs` any time a `@freezed` model changes.

`--web-port` is pinned because the backend's CORS policy allowlists a specific origin (`http://localhost:5001`) — `flutter run -d chrome` picks a random port otherwise, which breaks CORS on every relaunch. If port 5001 is ever busy, pick a different free port and update `WithOrigins( ... )` in the backend's `Program.cs` to match, then restart the backend (CORS middleware requires a real restart, not hot reload).

Available run targets in this environment: Chrome, Edge, Windows desktop (no Android/iOS emulator configured yet — run `flutter emulators` to check on your machine).

15. What this gives you

- A premium, glassmorphic login screen (gradient background, blurred glass card, floating-label fields, animated gradient CTA) calling `POST /api/Auth/login`.
- Real-time client-side validation (plain user ID pattern, required non-empty password) that only surfaces errors after a field is touched.
- Token stored securely and auto-attached as `Bearer <token>` on every subsequent request via the Dio interceptor; remember-me flag persisted alongside it.
- Auth-guarded routing: `go_router` redirects unauthenticated users to `/login` and authenticated users away from it, reactively (no manual `Navigator.push` after login).
- Biometric unlock UI wired to `local_auth`, safely inert on web/desktop.
- A social login row (Google/Apple/Microsoft) that's fully built and styled but currently commented out on the screen, and explicitly not wired to real OAuth — calling `loginWithProvider` surfaces a clear "not configured" message instead of silently failing or faking success.
- A responsive post-login shell (side rail $\geq 900\text{px}$, bottom nav bar + scan FAB below) wrapping a KPI dashboard, a shipments list/detail/scan flow, an alerts feed, and a profile/logout screen — all currently rendering from empty/placeholder data, ready to wire to real backend endpoints one feature at a time.
- A repeatable pattern for every new feature: `data/models` (JSON mapping) $\rightarrow$ `domain/entities` (pure business objects) $\rightarrow$ `presentation` (Riverpod provider(s) + screen) — `auth/` is the fully-wired reference; `shipments/`, `notifications/`, and `profile/` show the same shape mid-build, UI first.

16. Open items / next steps

- [x] Confirmed actual backend login response shape (flat, no `user` object, no `id` / `email`, nullable `token`) — `LoginResponseModel` / `AuthRepository` updated to match.
- [x] Built out the authenticated app shell (responsive side rail / bottom nav bar) and a real home dashboard (KPI grid + active containers).
- [x] Added shipments (list/detail/scan), notifications, and profile screens — currently UI-only, rendering empty/placeholder data (see §10–§12).
- [] Backend doesn't issue a bearer token on login yet — once it does, confirm the field name (`LoginResponseModel.token` currently expects `"token"`) and remove the `if (loginResponse.token != null)` guard in `AuthRepository.login`.
- [] Add token refresh handling in `DioClient` (401 interceptor) once tokens exist.
- [] Wire real OAuth for Google/Apple/Microsoft (client IDs, backend token exchange) and replace `AuthRepository.loginWithProvider`'s `UnimplementedError`.
- [] Implement `/auth/forgot-password` flow (`login_screen.dart`'s "Forgot password?" currently just shows a snackbar).
- [] Replace placeholder brand glyphs in `brand_glyphs.dart` with official logo assets before shipping.
- [] Add registration feature (`Register` button currently a snackbar stub).
- [] Add `data/` layers (repository + provider) for shipments and notifications, replacing the empty/hardcoded lists currently passed into their screens.

- [] Wire a real camera plugin (e.g. `mobile_scanner`) into `ScanScreen`'s `_ScannerFrame`, currently a stubbed placeholder.
- [] Set up flavors (dev/staging/prod) once staging/prod backend URLs exist.
- [] Test biometric login on a real Android/iOS device or emulator (untestable on Chrome/Windows — `BiometricService.isSupportedPlatform` is `false` there).
- [] Expand `test/` coverage beyond the login screen smoke test (form validation edge cases, router redirect behavior, repository error mapping).